

Docker and Gitlab-CI/CD



What is Docker

- Docker is a software platform that simplifies the process of building, running, managing and distributing applications.
- It virtualises the operating system of the computer on which it is installed, enabling the efficient deployment of containerised applications.
- The first edition of Docker was released in 2013, revolutionising how developers create and deploy applications.
- Docker is developed using the GO programming language, making it highly efficient and scalable.

Context

- Hosting multiple web apps on a single server
- Different frameworks (e.g. Node.js, Spring, Flask), different versions of the frameworks, and different dependencies used by each application
- Inability to have multiple versions of Node.js, Java, Python, etc., installed on the same machine

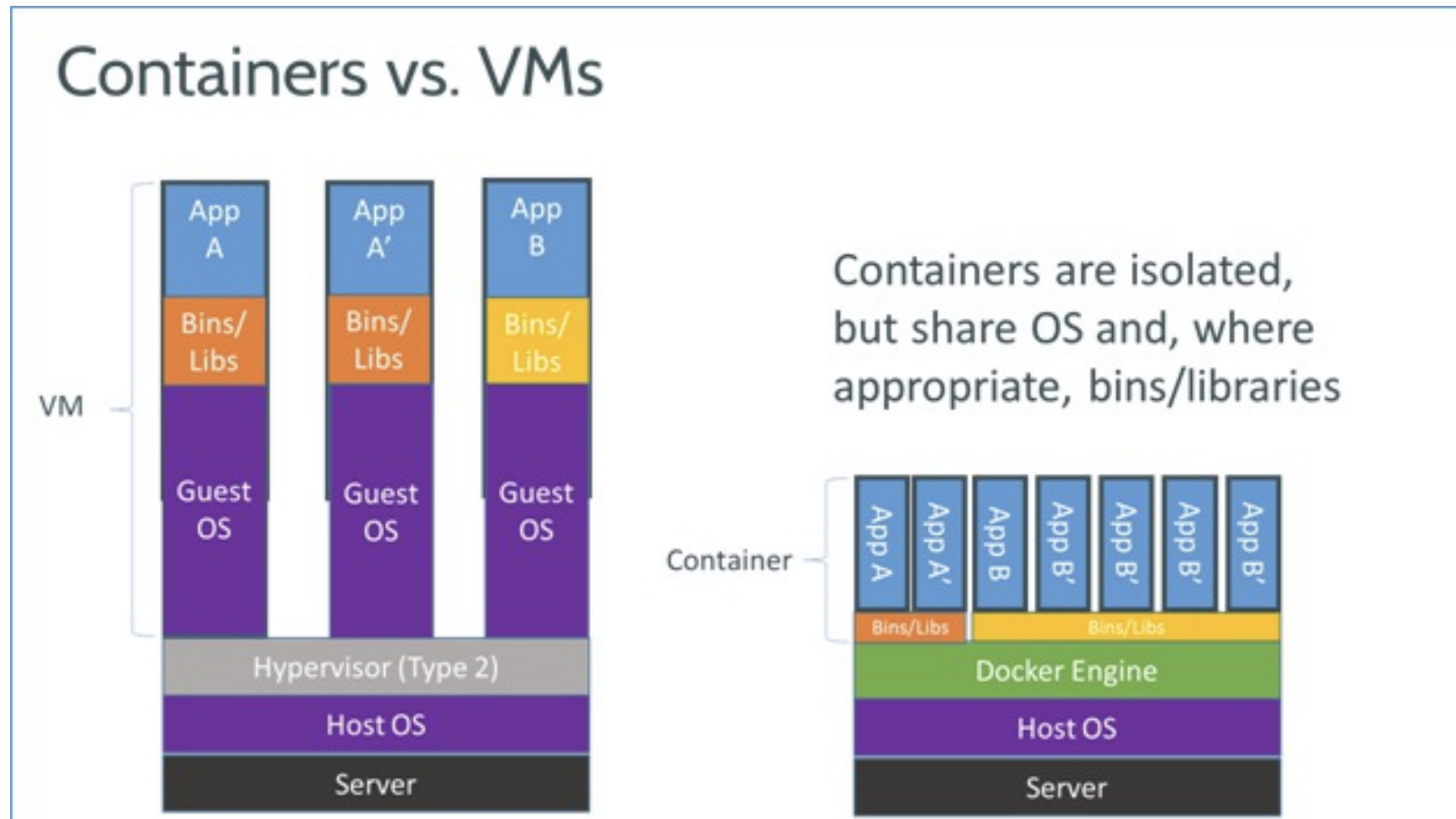
A solution without Docker

- Hosting on multiple physical machines
- Hosting on a single machine with multiple virtual machines
- High costs associated with procuring and maintaining hardware

A solution with Docker

- Docker Host and Containers
- Logical entities created by Docker Host
- Virtual copy of process table, network interface(s), and file system mount point(s)
- No operating system in the container
- Kernel of host operating system shared across all containers

Docker vs. VM



Docker Networking

- **Bridge Network:** Default network type. Containers on the same bridge network can communicate with each other using container names.
- **Host Network:** The container shares the host's network stack directly. Useful for performance but less isolation.
- **Overlay Network:** Enables container communication across multiple Docker hosts, mainly in Docker Swarm mode.

Advantages of Using Docker

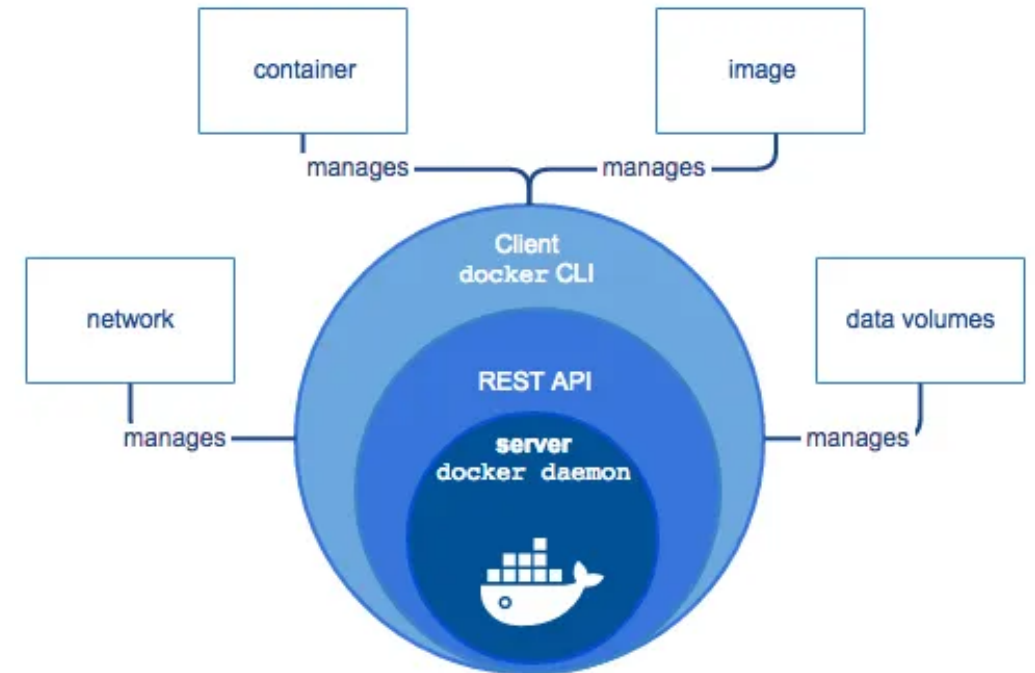
- Containers share the kernel of the host OS
- Multiple containers with different requirements can run on the same host
- Eliminates the need for multiple physical or virtual machines
- Docker allows multiple applications with different requirements to be hosted on the same host.
- Containers are generally small and consume very little disk space.
- Containers are more robust and boot up quickly compared to virtual machines.
- Docker is less demanding on hardware, reducing costs for users.

Disadvantages of Using Docker

- Decreased performance due to resource sharing
- The learning curve for new users
- Limited support for some operating systems
- Docker cannot host applications with different operating system requirements on the same host.
- This means applications with different OS requirements must be hosted on separate Docker Hosts.
- For example, Linux and Windows-based applications cannot be hosted on the same Docker Host.

Docker Engine

- Docker Engine is a critical part of the Docker system.
- Docker Engine manages the Docker platform's overall operations.
- It comprises three main components:
 - the Server
 - REST API
 - Client



Docker Engine Components

- The Server is the primary module of Docker Engine, which operates a daemon named dockerd.
- dockerd handles creating and managing Docker Images, Containers, Networks, and Volumes.
- The REST API enables applications to interact with the Server and order it to execute their functions.

Working with Docker Engine

- The Client is a command-line interface that lets users communicate with Docker.
- It allows users to use commands to communicate with Docker and Docker Engine.
- The Client streamlines the interactions with the Docker Engine for users.

Docker Terminology

- **Docker Image**

- A Docker Image is a template containing the application and all dependencies required to run that application on Docker. It serves as a blueprint for Docker containers.
- Images are static. They are snapshots of an application and its environment.

- **Docker Container**

- A Docker Container is a logical entity, essentially a running instance of a Docker Image.
- Containers are dynamic. They are the actual running instances of images, executing the application in a Docker environment.

Underlying Linux Technologies

- Docker leverages existing Linux features like overlay filesystems, cgroups (Control Groups), network namespaces, and binding mounts. These capabilities are native to Linux and can be manipulated directly using bash commands.
- Docker simplifies the utilisation of these complex Linux features by encapsulating them into a more user-friendly and accessible format through the concept of a "Docker Image." This innovation significantly streamlines the deployment and management of applications.

Docker Hub

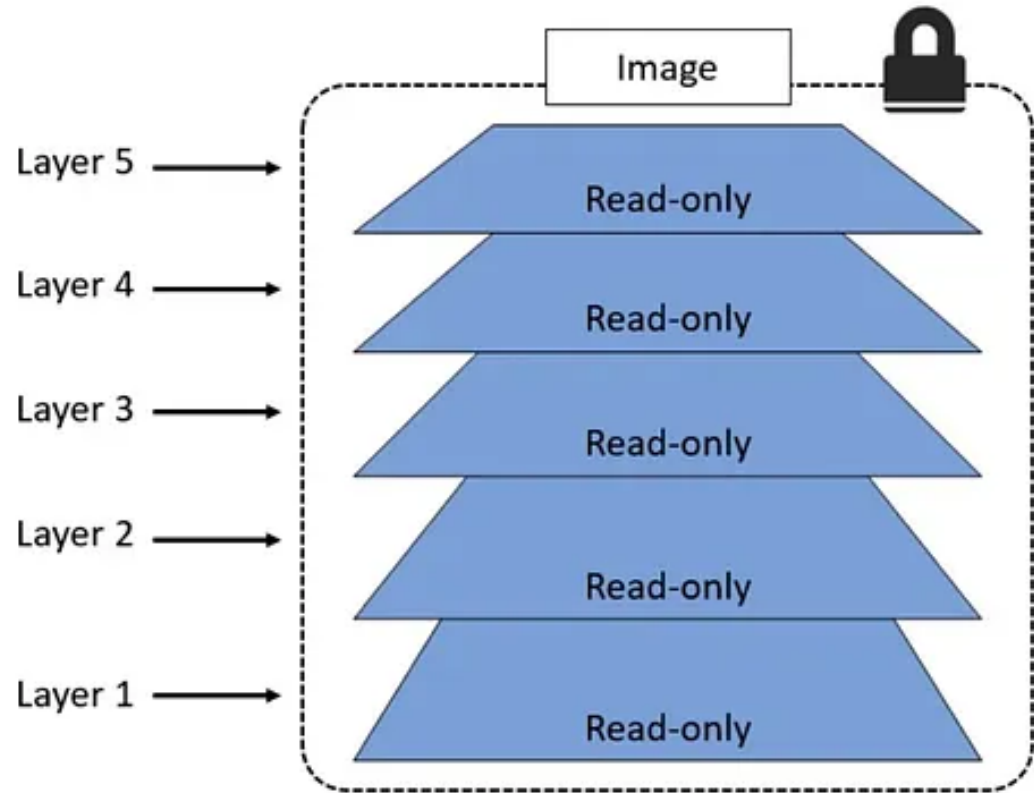
- An online repository for Docker Images
- Allows storage and distribution of custom images
- Option to make images either public or private
- Free users are limited to only one private image
- Paid subscription plan necessary for multiple private images

GitHub Container Registry (GHCR)

- GitHub Container Registry (GHCR) is a new feature that enables GitHub users to host and manage Docker images.
- It essentially allows users to store their container images in a central location, making it easier to manage and deploy their application code.
- GHCR provides users with a more streamlined workflow by allowing them to push and pull container images like any other code or file on GitHub.
- Free and unlimited storage: GHCR provides free and unlimited storage to all GitHub users, eliminating the need for users to set up and manage their container registries or pay for storage.

Building Docker Images

- Each instruction in the building file of the docker image creates a layer
- These layers are ReadOnly (R/O)
- At the end of the build, all R/O layers are stacked and stitched together to form an image
- This process is known as the Union File system

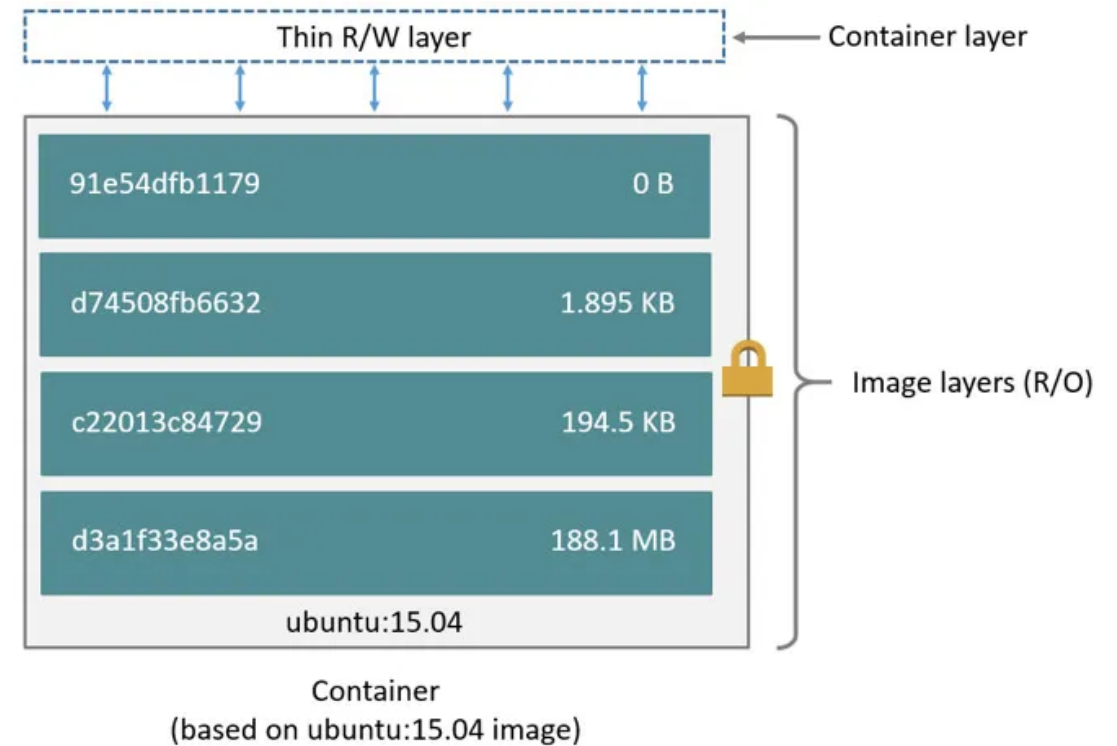


Docker Image Layers

- Docker image layers are immutable R/O layers.
- The Union File system allows for layers to be reused when unchanged and recreated when edited
- Changes made to the image instructions result in the creation of new layers, discarding existing ones

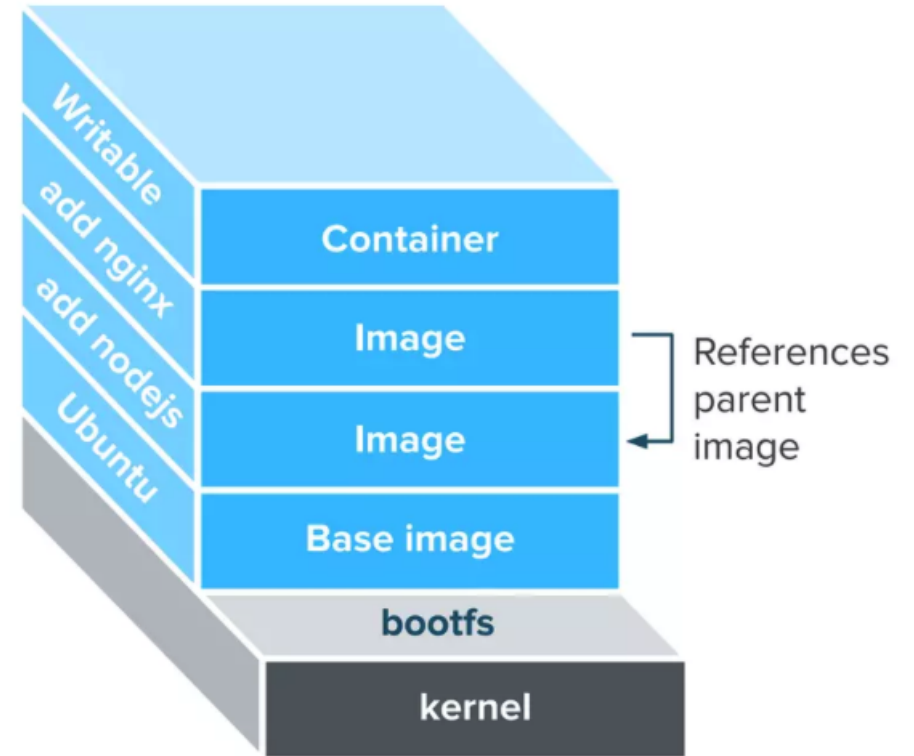
Creating Docker Containers

- Docker containers are made by creating a Read-Write (R/W) layer on top of a Docker image
- This layer is called the container layer
- Copy-On-Write (COW) is the process of copying and editing objects on this layer
- All changes to the container are stored on this layer



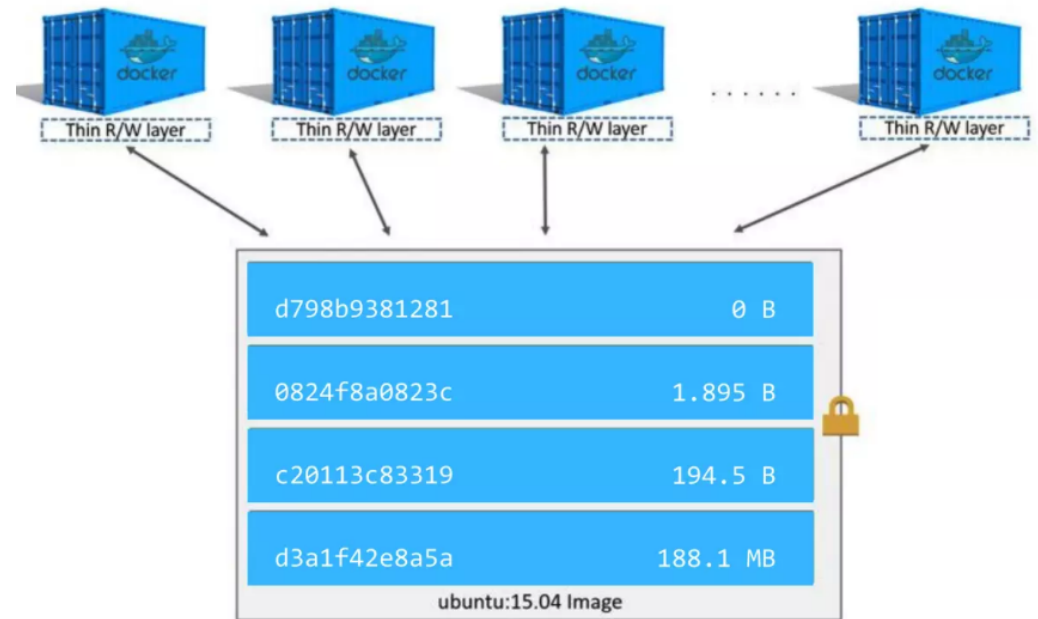
Docker Container Layers

- The container layer stays at the top of the image layers
- The image layers are immutable R/O layers, while the container layer is a detached R/W layer
- Changes made to the container layer do not affect the image layers
- The container layer is removed when the docker container is destroyed, resulting in the removal of all changes made to it



Multiple Same Container Instances

- Read-Only layers can be shared between any container that is started from the same image
- Writable layer is unique per container



How to store data

- **Volumes:** stored in the host filesystem and managed by Docker
 - **Bind mounts:** stored in the host filesystem and need a path
 - **Tmpfs mounts:** available only on Linux and not persisted on the host filesystem
-
- The data should be isolated from a container to leverage the benefits of adopting containerisation.

Docker Volumes Practical Example

- Data survives container recreation or crash.

```
1  services:
2    db:
3      image: mysql
4      volumes:
5        - db_data:/var/lib/mysql
6
7  volumes:
8    db_data:
```

Dockerfile

- A Dockerfile is a text file that contains instructions to build a Docker image.
- It specifies the base image, the application code and its dependencies, and the commands to be executed to set up the environment and run the application.
- The Docker engine uses the Dockerfile to automate the building of the image, which can then be used to create and run Docker containers that isolate the application and its dependencies from the host system.
- This enables the application to be easily deployed across different environments, platforms, and infrastructure.

Dockerfile Example

```
FROM node:latest
```

Use the most recent official Node.js image as the base image for this container.

```
WORKDIR /app
```

Create a new directory inside the container called /app and set it as the working directory for subsequent commands.

```
COPY . .
```

Copy all source files in the container (node modules are excluded)

```
RUN npm install
```

Install the app's dependencies inside the container based on the package.json file.

```
EXPOSE 3001
```

Expose port 3001 on the container as the one where the image will run

```
CMD ["npm", "start"]
```

Start the image using the script defined for starting the application

Multi-stage builds in Dockerfile

- Optimizes Docker images by separating build and runtime stages.
- Reduces final image size.

```
1  # Build stage
2  FROM node:18 AS build
3  WORKDIR /app
4  COPY . .
5  RUN npm install && npm run build
6
7  # Production stage
8  FROM node:18-slim
9  WORKDIR /app
10 COPY --from=build /app/dist ./dist
11 CMD ["node", "dist/server.js"]
12
```

Building & Running the Docker Container

- This command will create a Docker container
 - `docker build -t my-app-image .`
- This command will run the Node.js app inside the container, expose it on port 3000, and mount the specified local folder /path/to/local/folder to the container under the /app directory.
 - `docker run -d -p 3000:3000 -v /path/to/local/folder:/app my-app-image`

Tagging a docker image

- A tag is a name + version reference for an image
- Required to push to a registry
- The tag determines where and under what name the image will be stored
- Format: `registry/username/image-name:tag` (default tag is `latest`)
- Tag during build:

```
docker build -t my-app:1.0.0 .
```
- Tag an existing image for push:

```
docker tag my-app ghcr.io/myusername/my-app:1.0.0
```

Pushing a Docker Image

- Steps:

- Tag the image with the full registry path
- Log in to the registry

```
docker login ghcr.io
```

- Push the image

```
docker push ghcr.io/myusername/my-app:1.0
```

- Important:

- You must have write access to the target repository
- The image becomes available for others (if public) or for deployment

Pulling a Docker Image

- Pull an image from a registry to your local system:

```
docker pull ghcr.io/myusername/my-app:1.0
```

- Automatically done if you `docker run` an image that doesn't exist locally

```
docker run -d mysql
```

- Docker automatically pulls the `mysql:latest` image from Docker Hub (if not already available on the current machine)
 - Runs the container in detached mode
- Common scenarios where pulling an image is required
 - To use official or public base images (e.g., MySQL, Node, Nginx)
 - To retrieve the latest version of an internally maintained image
 - To deploy containerized applications during automated CI/CD workflows

Docker Compose

- Docker Compose is used to define and manage multi-container Docker applications
- They can be defined both in yaml and JSON format
- The `docker-compose.yml` file is a configuration file that defines the following:
 - Services: A service is a container running the application. A `docker-compose.yml` file can contain multiple services, each representing a different application component.
 - Networks: Networks are used to allow containers to communicate with each other. A `docker-compose.yml` file can define one or more networks that are used by the services in the application.
 - Volumes: Volumes allow persistent data to be shared between containers or to be stored outside of the containers. A `docker-compose.yml` file can define volumes that are used by the services in the application.

Docker Compose Override

- Compose supports multiple configuration files.
- `docker-compose.override.yml` is automatically used when you run `docker compose up`.
- Useful to differentiate development and production setups.

Example:

- `docker-compose.yml`: base configuration.
- `docker-compose.override.yml`: adds volumes, debugging tools, environment variables.

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

Docker-compose.yml

- The docker-compose.yml file specifies the configuration for each service, including
 - the image to be used,
 - ports to expose,
 - environment variables,
- Docker Compose automatically creates and configures the necessary containers based on the definitions in the docker-compose.yml file.

Yaml files

- A YAML file is a human-readable data serialization format that is often used for configuration files and storing structured data.
- It stands for "YAML Ain't Markup Language" and is designed to be both easy to read and write for humans and easy to parse for machines. YAML files are often used in web development, DevOps, and software configuration management.
- They use indentation and colons to indicate a hierarchy of values and can include complex data structures such as lists, dictionaries, and nested objects.
- YAML files can be edited using text editors, and typically end with the extension ".yaml" or ".yml".

Yaml vs. JSON

- YAML is more readable and flexible for complex configurations compared to JSON
 - Cleaner syntax (no {}, [], ")
 - Supports comments with #
 - Allows anchors and aliases (&name, *name) to reuse values
 - Easier to write multi-line strings (e.g. commands)
- Drawbacks:
 - Sensitive to indentation
 - More error-prone if not careful

Environment Variables

- Key–value pairs injected into a container's runtime environment
- Used to configure application behavior without changing code
- How to define them
 - Directly in Dockerfile using `ENV`
 - In `docker-compose.yml` under `environment`
 - (Optional) Using a `.env` file to centralize definitions
- Why use them
 - To separate config from code
 - To reuse values across services
 - To support different environments (dev, test, prod)

Environment Variables in Dockerfile

```
FROM node:latest

ENV SERVER_PORT=3001
ENV APP_ENV=production

WORKDIR /app
COPY . .
RUN npm install
EXPOSE ${SERVER_PORT}
CMD ["npm", "start"]
```

- ENV sets default values used during image build and container runtime
- `${SERVER_PORT}` is used in EXPOSE, but can be overridden in Compose
- You can use these variables in your application code as well (e.g., `process.env.SERVER_PORT` in Node.js)

Environment Variables in Docker Compose

- Pass values to containers:

```
services:  
  frontend:  
    image: app-frontend  
    container_name: app-fe  
    environment:  
      SERVER_HOST: localhost  
      SERVER_PORT: 5000  
      APP_PORT: 5173  
    ports:  
      - "5173:5173"
```

- Compose-defined variables override those in the Dockerfile
- Compose-defined variables are injected at runtime and readable from code
- Useful for environment-specific customization
- Anchors help maintain consistency across services

Secrets Management in Docker

- Environment variables expose sensitive data.
- For production, it is possible to use **Docker Secrets**.
- Secrets are encrypted and only available to containers that need them.
- Secrets are mounted as a file in `/run/secrets/<secret_name>` inside the container.

```
services:
  myapp:
    image: myapp:latest
    secrets:
      - my_secret
secrets:
  my_secret:
    file: ./my_secret.txt
```

Reusable constants with YAML anchors

- Anchors (&name) define reusable values
- Aliases (*name) inject the defined value
- Useful for keeping configurations consistent across services
- Defining constants with anchors

```
x-constants:  
  backend_host: &backend_host localhost  
  backend_port: &backend_port 5000  
  frontend_port: &frontend_port 5173
```

- Referencing them with aliases

```
environment:  
  SERVER_HOST: *backend_host  
  SERVER_PORT: *backend_port
```

Limitations of anchors in Docker Compose

- Anchors can only replace entire scalar values
- Anchors cannot be used inside strings
- Use anchors only where the full value is expected
 - e.g., for ports and volumes, values must be written explicitly
- Not Valid:

```
environment:
```

```
  APP_PORT: *frontend_port
```

```
...
```

```
ports:
```

```
- "*frontend_port:5173" # Invalid: partial  
interpolation is not supported
```

Example of Docker-compose.yml

This a real docker-compose file

It defines an image for the db, one for the webapp, a couple of volumes for the database and the crm data to ensure persistence

```
1  version: '3.7'
2
3  services:
4    db:
5      image: mariadb:10.6
6      restart: always
7      environment:
8        MARIADB_ROOT_PASSWORD: sssssss
9        MARIADB_DATABASE: ssss
10       MARIADB_USER: sssss
11       MARIADB_PASSWORD: sssss
12     volumes:
13       - db_data:/var/lib/mysql
14
15     espocrm:
16       image: espocrm/espocrm:latest
17       restart: always
18       ports:
19         - "18080:80"
20       environment:
21         - DATABASE_HOST=db
22         - DATABASE_PORT=3306
23         - DATABASE_NAME=ssss
24         - DATABASE_USER=ssss
25         - DATABASE_PASSWORD=ssss
26         - SITE_URL=https://www.mysite.com/crm
27         - PUBLIC_URL_PATH=/crm
28       volumes:
29         - espocrm_data:/var/www/html
30
31     volumes:
32       db_data:
33       espocrm_data:
```

Docker usage

- You can use Docker to test the application in a secure environment without needing any dependency or risking compatibility issues.

- You can launch it with:

```
docker compose up -d
```

- Or stop it with:

```
docker compose stop
```

Gitlab CI/CD



CI – Continuous Integration

- It is a software development methodology that delivers high-quality software frequently and efficiently.
- CI continuously integrates code changes into a shared repository, followed by automated tests and builds.
- With CI, developers can detect and fix errors early in the development cycle, reducing the impact of bugs on the final product.

CD – Continuous Delivery/Deployment

- CD automates the entire software delivery process, from building to testing to deployment to production.
- With CD, developers can release software to end-users with high frequency and confidence.

Continuous Delivery vs Continuous Deployment

- **Continuous Delivery:**

- The pipeline ends with a manual approval step before deploying to production.
- Human validation is possible before going live.

- **Continuous Deployment:**

- No human intervention.
- Every change that passes automated tests is immediately deployed to production.

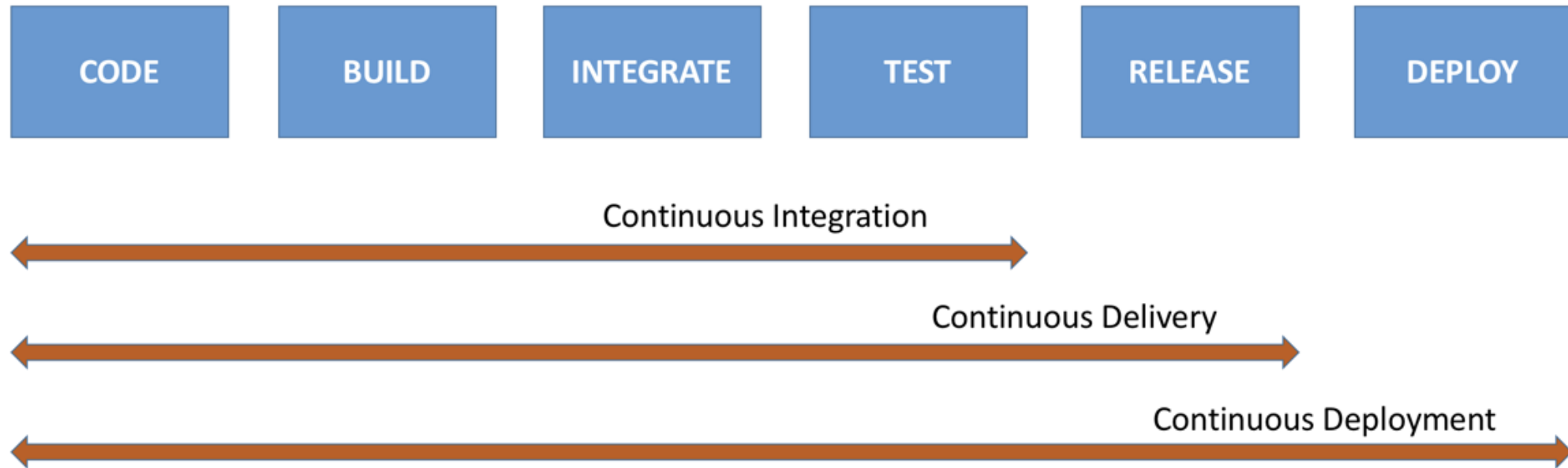
- **Key difference:** Continuous Deployment = Fully Automated Release.

CI/CD: Benefits and Workflow Stages

- CI/CD helps to increase the speed of software delivery, enhance collaboration among development teams, and improve software quality. It also helps to reduce the overall risk associated with software development.
- The CI/CD workflow consists of several stages:
 - code integration,
 - automated build,
 - testing, and
 - deployment.
- The workflow ensures that each code change is thoroughly tested and validated before it is released to production.

Release Pipeline

RELEASE PIPELINE



Gitlab CI/CD pipelines

- GitLab CI/CD pipelines allow developers to create, orchestrate, and automate their CI/CD processes within their GitLab environment.
- Pipelines include stages such as build, test, deploy, and release.
- GitLab CI/CD pipelines are configured via YAML files called ".gitlab-ci.yml".

GitLab CI/CD Artifacts and Cache

- **Artifacts:** Files generated during the pipeline (e.g., test reports, binaries) and saved for later stages or downloads.
- **Cache:** Reusable dependencies to speed up future pipeline runs (e.g., node_modules, Maven repo).

```
1  job:
2    script:
3      - npm install
4    cache:
5      paths:
6        - node_modules/
7    artifacts:
8      paths:
9        - build/
10
```

.gitlab-ci.yml

stages:

- test

stages section specifies the stages that the pipeline will run,
and in this case, there is only one stage, the test stage.

before_script:

- cd code/server
- npm install

before_script section specifies the commands that will run before each job,
switching to the code directory and installing dependencies using npm.

test_server:

stage: test

image: node:latest

script:

- set NODE_ENV=test
- npm test -testPathPattern='(test_unit|test_integration)'

script section specifies the command to run for the job,
which is running the npm test command on the unit and integration test suites.

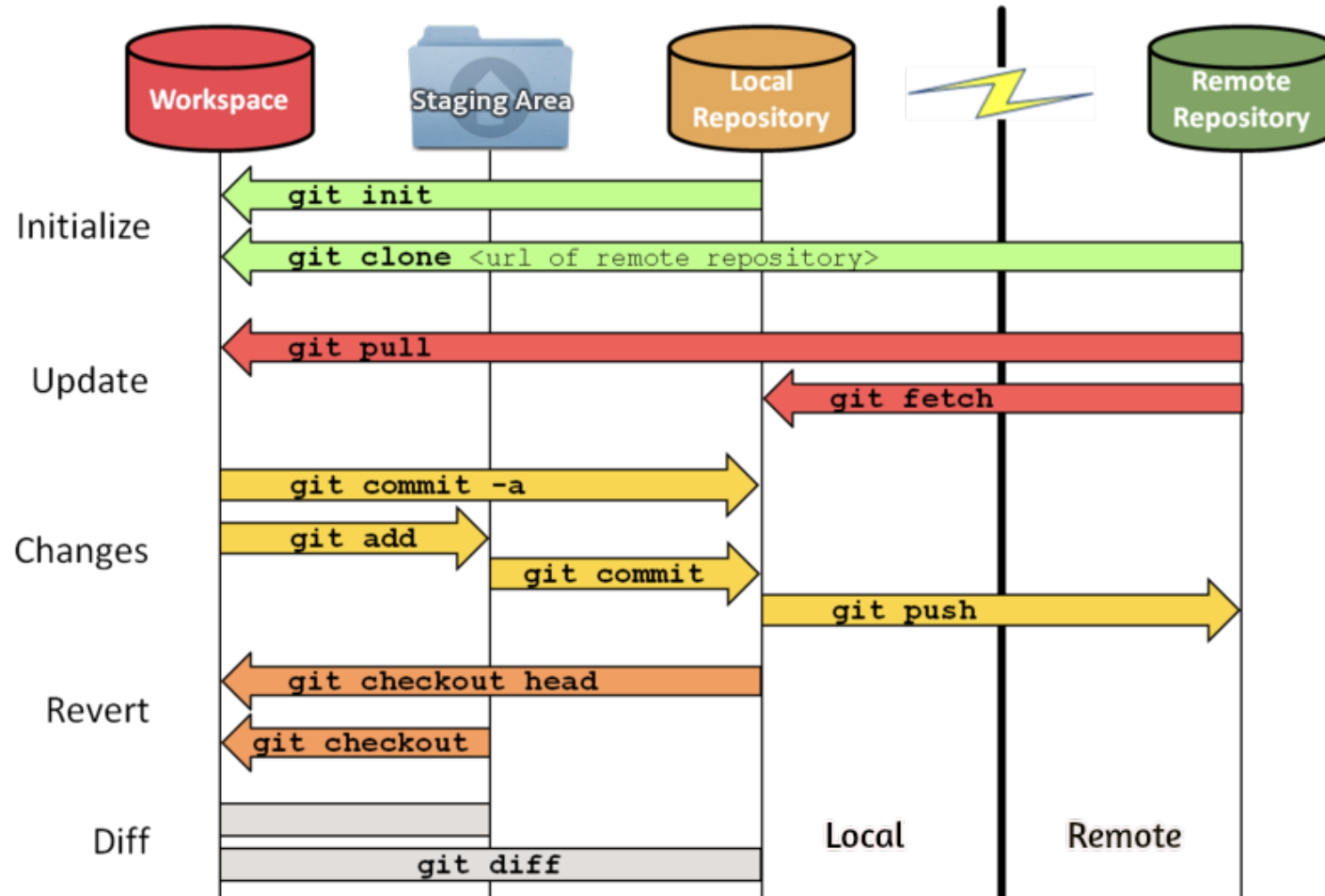
rules:

- if: '\$CI_COMMIT_BRANCH == "main"'

rules section specifies when the job should run.

In this case, the job will only run if the branch being pushed to is main

Git Workflow

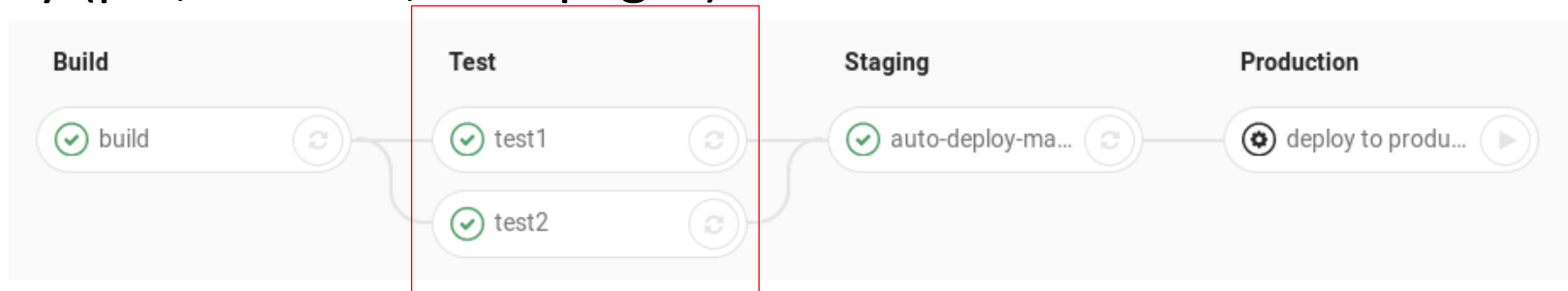


CI/CD Jobs Triggering

- By default, CI pipelines are triggered by `git push`
- CI jobs can be run on some specific branches only
 - In our project, it runs in the `main` branch
 - It is possible to have it run on a branch named `testdelivery` to check the results of your tests on the CI environment. Keep in mind that the machine needs to handle all requests, so use this functionality sparingly
 - **do NOT use `testdelivery` as the development branch where you make all your commits, use it EXCLUSIVELY for testing the CI pipeline!**
- external trigger (POST), useful for multi-project CI
- scheduled trigger

CI/CD Pipelines

- At every « `git push` », a script (`.gitlab-ci.yml`) is executed
- It launch jobs in sequence and/or in parallel (pipelines)
 - Build
 - **Test**
 - Deploy (pdf, binaries, web pages)



- Jobs are executed by « Runners » that require some set-up work
 - We have a pre-configured runner for our project. It's quite powerful, but it can be overwhelmed when we are close to the deadline!
 - That's why ci/cd is run only in the `main` branch

Example

 Update .gitlab-ci.yml file
Giacomo Garaccione authored 7 minutes ago

Passed
✓
caacd58e 

Commit caacd58e  authored 7 minutes ago by  Giacomo Garaccione Browse files Options ▾

Update .gitlab-ci.yml file

 parent [ac67f332](#)

 Branches v2-base

 No related merge requests found

 Pipeline [#20471](#) passed with stage  in 57 seconds

Example

Search job log

48

PASS

test_unit/dao_tests/user.test.ts

49

✓ It should resolve true (5 ms)

42

-----|-----|-----|-----|-----|

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	46.08	8.88	14.76	46.36	
server	85.71	0	0	85.71	
index.ts	85.71	0	0	85.71	18-19
server/src	69.56	0	22.22	68.88	
helper.ts	27.27	0	25	27.27	17-25,34
routes.ts	100	100	100	100	
utilities.ts	40	100	0	33.33	14-38
server/src/components	50	100	50	45.45	
user.ts	50	100	50	45.45	22-27
server/src/controllers	33.33	100	8.77	33.33	
cartController.ts	27.27	100	5.88	27.27	23-75
productController.ts	30	100	6.66	30	24-74
reviewController.ts	37.5	100	9.09	37.5	19-47
userController.ts	40	100	14.28	40	32-77
server/src/dao	34.04	0	33.33	36.36	
cartDAO.ts	100	100	100	100	
productDAO.ts	100	100	100	100	
reviewDAO.ts	100	100	100	100	
userDAO.ts	29.54	0	33.33	31.7	19-41,64-65,78-98
server/src/db	87.5	40	100	100	
db.ts	87.5	40	100	100	16-23
server/src/errors	40	100	0	27.58	
userError.ts	40	100	0	27.58	17-96
server/src/routers	42.72	0	15.32	44.28	
auth.ts	38.18	0	9.52	36.36	50-56,67,77-169
cartRoutes.ts	42.85	100	11.11	42.85	53-58,71-74,86-89,100-102,113-116,128-130,140-142,152-154
productRoutes.ts	43.58	100	12.5	43.58	61-63,77-79,93-97,112-114,128-130,140-142,153-155
reviewRoutes.ts	46.87	100	16.66	46.87	39-43,55-57,68-72,84-86,96-98
userRoutes.ts	51.85	100	23.52	50.94	62,73-75,86-88,99-101,112-114,124-126,142-144,193-195,205-207,217

-----|-----|-----|-----|-----|

75

Test Suites: 3 passed, 3 total

76

Tests: 3 passed, 3 total

77

Snapshots: 0 total

78

Time: 6.52 s

79

Ran all test suites.

80

Cleaning up project directory and file based variables

81

Job succeeded

Duration: 57 seconds

Finished: 5 minutes ago

Queued: 2 seconds

Timeout: 1h (from project)

Runner: #2 (HsLLzEA7z) harpo

Commit caacd58e

Update .gitlab-ci.yml file

Pipeline #20471 Passed for v2-base

test

Related jobs

→ test_server

Reporting problems

- If you notice that our code has some problems open an issue in the **root project**. If possible, describe the steps for replicating the issue
 - <https://git-softeng.polito.it/se2024-25/geocontrol/-/issues>
- You can use the issues of your project to communicate problems between team members

New Issue

Title (required)

Add [description templates](#) to help your contributors to communicate effectively!

Type ?

Description

Write Preview

Write a description or drag your files here...

Supports [Markdown](#). For quick actions, type `/`.

☐ This issue is confidential and should only be visible to team members with at least Reporter a

Assignee

 [Assign to me](#)

Select assignee ✕

🔍

✓ Unassigned

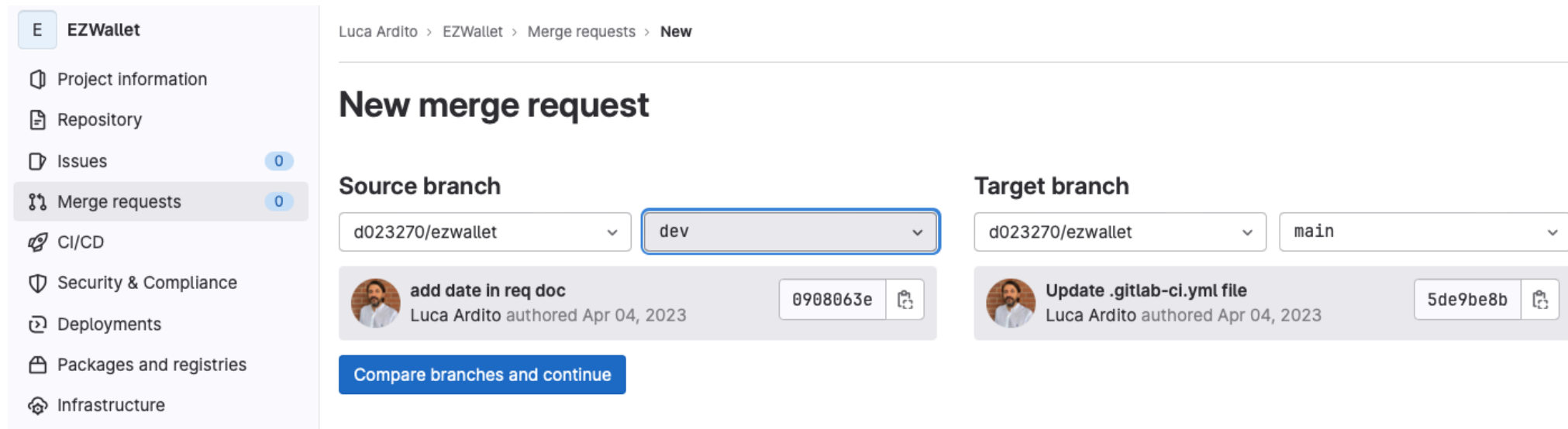
 **Luca Ardito**
@d023270

 **Giacomo Garaccione**
@s233694

 **Maurizio Morisio**
@d001921

How to deliver?

- Project deliveries will be on the main branch of **your** GeoControl project.
- Since you are not allowed to push on the main branch, you will have to **open a merge request**
- If you work on a branch called **dev**, you ask to merge the content of the dev branch on **main** with a merge request



The screenshot shows the 'New merge request' page in GitLab for the 'EZWallet' project. The left sidebar contains navigation links: Project information, Repository, Issues (0), Merge requests (0), CI/CD, Security & Compliance, Deployments, Packages and registries, and Infrastructure. The main content area has a breadcrumb trail: Luca Ardito > EZWallet > Merge requests > New. The title is 'New merge request'. Below this, there are two sections: 'Source branch' and 'Target branch'. The 'Source branch' section shows a dropdown for 'd023270/ezwallet' and a selected branch 'dev'. Below this is a commit preview for 'add date in req doc' by Luca Ardito, authored Apr 04, 2023, with commit hash 0908063e. The 'Target branch' section shows a dropdown for 'd023270/ezwallet' and a selected branch 'main'. Below this is a commit preview for 'Update .gitlab-ci.yml file' by Luca Ardito, authored Apr 04, 2023, with commit hash 5de9be8b. At the bottom of the source branch section is a blue button labeled 'Compare branches and continue'.

How to deliver?

- After clicking on **compare branches and continue**, you will see this page in which you can add some details about your merge request
- Check the “delete source branch” option to start with a new branch directly from the updated main branch after the merge request acceptance.

EZWallet

- Project information
- Repository
- Issues 0
- Merge requests 0
- CI/CD
- Security & Compliance
- Deployments
- Packages and registries
- Infrastructure
- Monitor
- Analytics
- Wiki
- Snippets
- Settings

New merge request

From `dev` into `main` [Change branches](#)

Title (required)

28/04 documents delivery

☐ Mark as draft
Drafts cannot be merged until marked ready.

Add [description templates](#) to help your contributors to communicate effectively!

Description

Write Preview **B** **I**

add date in req doc

Supports [Markdown](#). For [quick actions](#), type `/`.

Assignee

Unassigned [Assign to me](#)

Reviewer

Unassigned

Milestone

Select milestone

Labels

Labels

Merge options

☒ Delete source branch when merge request is accepted.

☐ Squash commits when merge request is accepted. [?](#)

[Create merge request](#) [Cancel](#)

Commits 1 **Changes** 1

04 Apr, 2023 1 commit

add date in req doc
Luca Ardito authored 2 weeks ago

CI/CD pipeline

- When the teachers accept the merge requests, the CI/CD is triggered

SE 2022-23 > EZWallet > Pipelines > #18046



Pipeline #18046 triggered 2 weeks ago by Luca Ardito

Cancel running

Delete

Update .gitlab-ci.yml file

🕒 2 jobs for `main` in 58 seconds (queued for 1 second)



latest



Sde9be8b

🔗 No related merge requests found.

Pipeline Needs Jobs 2 Tests 0

test



test



SE 2022-23 > EZWallet > Jobs > #23019



Job test triggered just now by Luca Ardito

Search job log



```
1 Running with gitlab-runner 15.10.0 (456e3482)
2   on harpo HsLLzEA7, system ID: r_eavLH4tTR8PD
3   Preparing the "docker" executor 00:11
4     Using Docker executor with image node:14 ...
5     Starting service mongo:4.4 ...
6     Using locally found image version due to "if-not-present" pull policy
7     Using docker image sha256:afef20758bd3d32ba19f0c36cabe79973a98279ca35a1d7e2260cb
  ca1e5ba395 for mongo:4.4 with digest mongo@sha256:7fb082eda5d1096c7749bd3890f0e23
  ad99fc5059902b5e47df8873d059c749a ...
8     Waiting for services to be up and running (timeout 30 seconds)...
9     Using locally found image version due to "if-not-present" pull policy
10    Using docker image sha256:cedb9f0d0498e80fa9e493372426e7dee95e03474a6ef544faf547
  f9726bf7b5 for node:14 with digest node@sha256:481a78885f7a83f86fcb3ac48dab9bede6
  51851d1b250650c17645a988902728 ...
12    Preparing environment 00:02
13    Running on runner-hsllzea7-project-6469-concurrent-0 via 739bdead498a...
15    Getting source from Git repository
16    Fetching changes with git depth set to 20...
17    Reinitialized existing Git repository in /builds/se-2022-23/ezwallet/.git/
...

```

Possible timeout issue

- It is possible for the CI/CD pipeline to timeout. This is because, in a few minutes, 130 pipelines are instantiated all at once against the merge requests of all groups.
- The machine is powerful, but it is only one, so it may not be able to handle all the requests within the 60-minute timeout.
- The teachers will check the jobs and restart them if this condition occurs.